

DSA w/ CISCO

Part 1 of Sets and Dictionaries

Fabiola C. Villanueva

Friday, October 10, 2025 | 6:00–9:00 PM

[\[DSA by CISCO\] Sets & Dictionaries Tutorial PPT by Bobby](#)
[MIRO Board Link](#)

Sets

- Overview
- Functions and Operations
- Implementation
 - Bit-Vector
 - Computer Word

Dictionaries

- Overview
- Functions and Operations
- Implementation
 - Open Dictionaries
 - Close Dictionaries
 - Collision Management

On a personal note...

In a world where you can be whoever you want to be, **be kind**.

Sets: An Overview

What is a set?

- A set is a collection of **unique elements in which the order of these elements do not matter** (*for as long as they are a unique collection!*)
- Recall where you have previously heard of sets, before even stepping foot in DSA. It typically mentions terms such as finite set, subset, superset, universal set, etc.
 - I want us to focus on the terms **universal** and **subsets** for now to better understand the discussion by relating it what we already know
- By definition, a subset's number of elements can only be less than or equal to the number of elements in a universal set. Whereas, by the same logic, a universal set's number of elements can only be greater than or equal to the number of its subsets – your universal set essentially defines the maximum number of elements for each of its subsets.

Example:

- Say that I have a universal set of 10 elements, how many elements are my subsets allowed to have? (*10 or less*) – *dependent to the largest element in the universal set*
- How about when I have a subset of 10 elements, how many elements can my universal set possibly have? (*10 or more*)

Functions and Operations

Now...

- Previously, when we've mentioned the term ADT, we typically refer to the possible implementations we represent in memory.
- Feel free to think of it as, "if something is ADT, then there are certain functions and operations enabled to be performed UNLESS said otherwise"
- If we say ADT, then its functions and operations typically include your CRUD operations:

	ADT List	Stacks	Queues
C	Create, initialize, add	Initialize, push	Initialize, enqueue
R	Read or display	Read or display top	Read or display front
U	Update or replace		
D	Delete or remove	Pop	Dequeue

Additionally...

- When we say SETS, we refer to another way of implementing and representing data in memory
- You may think of it as, "if something is a SET, then are certain functions and operations enabled to be performed UNLESS said otherwise"
- If we say SET, then its included functions and operations are typically UID:
 - Let there be a SET A = {3, 2, 8, 9, 5} and SET B = {9, 1, 3, 10}

Union	Producing a new set that contains all unique elements found in both A and B. Hence $SET\ C = A \cup B$, then $C = \{3, 9, 2, 1, 8, 10, 5\}$
Intersection	Producing a new set that contains all common elements found from A and B. Hence $SET\ C = A \cap B$, then $C = \{3, 9\}$
Difference	Producing a new set that contains elements found in A but not in B. Hence $SET\ C = A - B$, then $C = \{2, 8, 5\}$

Therefore...

When we say and declare an ADT SET, we include and consider CRUD & UID functions when implementing.

Implementation

Speaking of “implementing”, previously...

- We’ve gotten our hands on array list, linked list, and cursor based – however, these implementations are for **lists**. Mind you we are now in **sets**.
- In addition to these existing implementations, we now introduce “set exclusive implementations” such as bit vector and computer.
- Let’s slowly walk through each one as to how sets are represented in memory or how they are implemented.

To clarify, Bit Vector can perform CRUD operations but since we now consider representing them as sets, we shall also treat them like sets by UID operations. *Same concept as to how a stack is simply an array in memory but we treat it with stack operations because that is how the specified data structure is supposed to be treated.*

Set Implementation: Bit-Vector

Now, let's start with this. The **best implementation of an ADT SET** shall depend on the size of its universal set and the operations to be performed.

- In other words, the size of the set shall be dependent to its universal set (OR the size of the subset shall **inherit** the size of its universal set)
- `sizeof(universalSet) == sizeof(subset)`: regardless the actual number of elements in the subset
- This is attained by a **BIT VECTOR**, which simply translates to "a one-dimensional array of binary digits"

Say you declare your one-dimensional array of binary digits – you may opt to set values to boolean or simply declare it as an integer array but only use values of ones and zeroes (boolean array).

```
#define SET_SIZE 8
int universalSet[SET_SIZE] = {0,1,2,3,4,5,6,7}
int subsetA[SET_SIZE] = {2,4,7}
```

Let's visualize the code above!

universalSet	1	1	1	1	1	1	1	1
	0	1	2	3	4	5	6	7

subsetA	0	0	1	0	1	0	0	1
	0	1	2	3	4	5	6	7

Questions (explain to the best of your ability):

- Why didn't the subset occupy the entire space?
 - *Because only elements in the subset are to be set to one whereas the rest shall be zero.*
- Why are the sizes for each subset still the same?
 - *Subsets inherit the SIZE of the universal set regardless of the number of elements – this is the nature for bit-vectors.*
- How do you determine the size of the universal set?
 - *The size of the universal set will be the largest number in the given set + 1 (you want the largest number to be an existing element in the bit vector)*

What really happens in a bit-vector or a boolean array?

- Given a set that is declared as a universal set, the size for your boolean array will be dependent on the largest number in that set + one
 - Adding one ensures that all numbers in the set each have a dedicated unique spot in the boolean array, including the largest number in the given universal set
- Your bit vector should have been initialized to zero upon creating the boolean array itself
- Values in the given set will be the indices in the bit vector that shall be set to true or 1
- Any subset from the universal set onwards shall inherit the size of the subset REGARDLESS the number of elements in the subset
- Hence, the largest element of a subset can only be less than or equal to the largest element from its universal set.

Time Complexity: $O(1)$ = isMember $O(1)$ = insertAt $O(1)$ = deleteAt

- Since a bit-vector is essentially just an array
- Accessing each element is simply through indexing, operations will be of $O(1)$ or constant
 - Unless operations involve traversing, such as combining sets
- Additionally, membership is checked to ensure that the N th bit is true if N is an element of the set

Pros	Cons
Constant-Time Membership Testing Checking if an element exists in the set (e.g., <code>is_member(x)</code>) is $O(1)$ because it involves a single bit access at index x .	Limited to Universal Set A bit vector can only represent elements within a predefined universal set. If the universal set grows (e.g., you need to include elements beyond 60), you must resize the bit vector, which can be costly or impractical.
Compact Representation for <u>Dense</u> Sets When the set is dense (i.e., many elements from the universal set are present), a bit vector is space-efficient since each element is represented/accommodated by a single bit.	Space Inefficiency for <u>Sparse</u> Sets As mentioned, bit vectors are not space-efficient for sparse sets (where most bits are 0). The memory usage is determined by the size of the universal set, not the actual number of elements in the set.
Predictable Memory Usage The memory required is fixed and proportional to the size of the universal set (e.g., one bit per element). This makes it predictable and easy to allocate.	Fixed Size Limitation The size of the bit vector is fixed at creation based on the universal set. If the universal set is very large (e.g., all 32-bit integers), the memory requirements become impractical (e.g., 2^{32} bits = 512 MB), even if the actual set is small.
Simplicity The implementation is straightforward, as it relies on basic bitwise operations and array indexing, making it easy to code and maintain.	No Support for Multiplicity (must be a unique) Bit vectors can only indicate presence or absence (1 or 0). They cannot represent multisets (where elements can appear multiple times) without additional structures.
Accommodates All Elements in Universal Set As you noted, a bit vector can represent any element in the universal set (e.g., 0 to 60) by reserving a bit for each possible element. This ensures no element is excluded as long as it's within the universal set's range.	Cache Inefficiency for Large Sets For very large universal sets, the bit vector may not fit in cache memory, leading to slower access times compared to more compact structures like hash tables for sparse sets.

Set Implementation: Computer Word

A bit-vector represents a set using an array of integers, where each element stores only 1s and 0s. If

the universal set is an array of 8 integers (or of size 8), then the memory occupied will be `[sizeof(int) * 8] bytes`. Hence, your universal set and its subset(s) will always have **32 bytes**, regardless of how many elements are used. Its fixed size makes it inefficient for sparse sets.

So how can we make sets be implemented as efficiently as possible?

- This is where we introduce one of our new implementations – **Computer Word**, or sometimes known as Data Word.
- A computer word in data structure refers to a standardized unit of data designed to be handled and processed through a single operation (UID operations)
- Note that the size for each computer word is dependent on your computer's processor architecture – varying from 8 bits to 64 bits or more

A **Computer Word**, in contrast, is a more compact representation where a single integer (or word) is used to represent a set. Each bit in the word corresponds to an element in the universal set, and the word size depends on the processor architecture (e.g., 8 bits for a char, 32 bits for an int on a 32-bit system, or 64 bits on a 64-bit system). This approach is space-efficient because a single word (e.g., 1 byte for a char) can represent multiple elements (e.g., 8 elements in a char), and operations like union, intersection, and complement are performed using fast bitwise operations (AND, OR, NOT, XOR).

Why use Computer Word?

- The context of sets for DSA implemented in computer word is significant as it directly impacts memory usage, performance, and efficiency when representing and manipulating data
- Previously, we have gone over how bit-vectors are of fixed sizes, dependent to its data type and number of elements
 - This time, though computer words are still of fix sizes, they don't occupy as much space as bit-vectors do, space efficiency wise in your computer's memory
- The significance of using computer words for sets lies in:
 - Space Efficiency: A single word can represent up to word_size elements (e.g., 32 elements in a 32-bit int), using far less memory than a bit-vector for small or medium-sized sets
 - Performance: Bitwise operations (AND, OR, XOR, NOT) are extremely fast, often executing in a single CPU cycle, making set operations like union or intersection highly efficient
 - Simplicity: No need for complex array management or dynamic memory allocation for small sets that fit within a word

With computer words, you can most definitely represent a unique element PER BIT of a word. How?

- A computer word can represent a unique element per bit, enabling efficient set operations
- When a set is small enough to fit within a single word (e.g., 32 bits for a 32-bit word), operations like union, intersection, or complement can be performed with a single bitwise operation (AND, OR, XOR, NOT) – **leveraging direct bit manipulation**
- A set is represented by an integer's bits, with each bit indicating the presence (1) or absence (0) of an element
- The total number of representable elements is the word size in bits (e.g., `sizeof(int) * 8` for an integer), allowing compact and fast set manipulation through bitwise arithmetic

Simply put!

We are representing unique elements as bits where the number of these bits can be derived or inherited from already existing datatypes such as unsigned/sign char, ints, long ints, etc. Say you want to represent 8 elements, then you may use the datatype char, a char is 1 byte == 8 bits. Hence, each element can be represented through each bit.

At the end of the day, you are able to represent each element by using only 1 byte or 8 bits memory. Unlike how you would a bit-vector, where how much space it occupies in memory is dependent on the number of elements multiplied to the size of the data type in itself.

For example, to represent 8 elements, you can use a single unsigned char (1 byte = 8 bits), where each bit represents one element. Compare this to a bit-vector, which might use 8 integers (32 bytes) for the same universal set, wasting memory if only a few elements are present. Each bit in the byte (8 bits) represents a unique element in a set, and the choice of signed vs. unsigned affects how values are interpreted and manipulated.

Signed Sets

- Range: -128 to 127 (1 byte = 8 bits, with the most significant bit as the sign bit).
- Sign Bit: The 8th bit (MSB) indicates the sign (0 for positive, 1 for negative).
- Behavior:
 - Values like SET x = 'A'; (ASCII 65) are stored as 01000001 (positive).
 - Negative values (e.g., -1) are stored in two's complement (e.g., 11111111 for -1).
 - Bitwise operations (e.g., &, ^) work on the bits but must account for the sign bit, which can affect arithmetic or comparisons.
- Use Case: Rarely used for sets, as negative values are less intuitive for representing set membership (bits as elements).

```
typedef signed char SET;  
SET x = 'A'; // ASCII 65 = 0100 0001  
SET y = -1;  // 1111 1111 (two's complement)
```

Unsigned Sets

- Range: 0 to 255 (1 byte = 8 bits, no sign bit).
- Sign Bit: None; all 8 bits represent the magnitude or set elements.
- Behavior:
 - Values like SET x = 'A'; (ASCII 65) are stored as 01000001.
 - Values like SET y = 88; (ASCII 'X') are stored as 01011000.
 - Bitwise operations are straightforward, as all bits represent elements without sign complications.
- Use Case: Preferred for sets, as each bit cleanly represents an element (0 = absent, 1 = present) in a universal set of up to 8 elements.

```
typedef unsigned char SET;  
SET x = 'A'; // ASCII 65 = 0100 0001 (bit 0 and bit 6 set)  
SET y = 88;  // // ASCII 'X' = 01011000 (bits 3, 4, 6 set)
```


Key differences:

Aspect	Signed Set	Unsigned Set
Range	-128 to 127	0 to 255
Sign Bit	Yes (MSB is sign)	No (all bits for magnitude)
Bit Interpretation	Sign bit complicates bitwise operations	Bits directly represent elements
User for Sets	Less common, sign bit may confuse	Ideal for set representation
Example SET x = 65	0100 0001 (positive)	01000001 (same, no sign)

Say that we want to implement the following into computer words...

```
typedef unsigned char SET;    // we declare a universal set here, alias "SET"

SET x = 'A';                  // subset x is initialized to the value 'A'
                               (ASCII value = 65, Binary value = 0100 0001)

SET y = 88;                   // subset y is initialized to the value 88
                               (ASCII symbol = 'X', Binary value = 0101 1000)
```

- How are sets declared and initialized as implemented in Computer Word?
 - SETS are declared by declaring your UNIVERSAL SET first – this is noticeably by your line of code

```
typedef unsigned char SET;
```

- Now per everytime you create a SET in your main function using the defined set above, those sets are inherently SUBSETS of the UNIVERSAL SET from "unsigned char SET"
 - These subsets inherits the following:
 - The size of your set in bytes
 - The number of unique elements you can represent in bits
 - It's un/signed state
 - Etc
- How does CRUD come into play?
- How does UID come into play?
 - Bitwise operations

Dictionaries: An Overview

What is a dictionary?

- A dictionary is a data structure that stores key-value pairs, allowing fast retrieval, insertion, update, and deletion of values based on unique keys.
- Often referred to as a hash map, map, or associative array. This is simply **another way to represent your data in memory** wherein **each unique element in the set has its corresponding hash-key value**. This hash-key value is typically derived from the given element wherein it is passed into a hash function to generate a hash-key. Your hash function shall only generate a hash key that corresponds to an existing space in your dictionary. This function may vary per specification such as generating a key by getting the sum of the element's name and the like.

The idea there is that, to determine a given element's position in the dictionary on where it must belong, a hash key must be generated out of the element itself and must return the position of where the element must be stored.

Hashing

Hashing is deriving or generating the key for a given element. This requires or provides constant time per operation implemented in dictionaries. Best case scenario for a hash function is when it provides the direct hash-key with zero collisions, whereas its worst case will be a time complexity proportional to the size of the set per operation.

Picture this: **a file cabinet analogy...**

When the hash function is invoked, it shall provide the key of which file drawer in the cabinet should an element (a file or folder) belongs to.

It at leasts returns either of the two things:

1. The exact location of the element,
2. The location on where you must start searching for the element, closest to its hash-key position.

The perfect hash function produces a unique key which makes collisions into the dictionary **non-existent**; it is not impossible but surely quite a challenge.

Example:

- Suppose a one-dimensional file cabinet, let's refer to each file cabinet as a bucket. Everytime I am to return/retrieve/search for a specific file.
 - I shall invoke the hash-function, input the my desired file's information such as file number/file name/etc depending on what information the hash function requests.
 - The hash function shall then return the exact bucket number of where my file is expected to be located.
 - In an **open dictionary**, the hash function will return the exact bucket number wearing the bucket can accommodate multiple files at once. This "disregards" the concept of collision as files with the same hash-keys simply share the file bucket. Collisions still occur; they're just resolved differently.
 - Whereas, in a **close dictionary**, the hash function may return the exact bucket number

but may not necessarily find the desired file in that bucket. If you are placing a file in the cabinet and the bucket at the returned hash-key is occupied, you must begin your search to the next available bucket (this is what you call **linear probing**, a collision management technique). However, if the bucket is available when the hash function returns the bucket number, then you may place the file directly to its hashed bucket number position.

Functions and Operations

A dictionary is a data structure that stores key-value pairs, where each unique key maps to a value. It's often called a hash map, map, or associative array. Dictionaries use a hash function to compute a key's position (index) in memory, enabling fast operations (typically $O(1)$ average time). Dictionaries support CRUD operations (Create, Read, Update, Delete) and additional set-like operations.

Operation	Description
Create	Add a new key-value pair to the dictionary. Compute the hash of the key and store the value at the resulting index. Handle collisions if needed.
Read (Get/Lookup/Display)	Retrieve the value associated with a key. Hash the key to find its index and return the value stored there.
Update	Change the value associated with an existing key. Hash the key, locate the index, and replace the old value with the new one.
Delete (Remove)	Remove a key-value pair from the dictionary. Hash the key, locate the index, and free the entry or mark it as empty.
Hash Function	Returns the key (index) of an element or value from the dictionary. When invoked in an open dictionary, it returns the exact index. When invoked in a close dictionary, it either returns the exact index or returns the index where you may begin your linear probing.

ADT Set Operations in Dictionaries

The following operations apply to both open and closed dictionaries, open and close.

Insert (elem x, set dict)	Adds an element in the dictionary (only if the element does not exist) by invoking the hash function to determine the position to insert the element.
Delete	Deletes an element in the set dictionary (only if the element exists) by invoking the hash function to determine which position the element is approximately located.
isMember	Returns TRUE (non-zero) if the element is in the set dictionary, otherwise FALSE (zero)

Implementation

An application you may observe hashing or dictionaries would be your primary and secondary keys in your database management systems. A dictionary in memory is typically represented as an array of buckets, where each bucket stores a key-value pair or a linked list (for collisions). The difference between an Open Dictionary and Close Dictionary simply lies in how they handle collisions.

- **Open dictionaries** use this term **“chaining”** for every time a synonym occurs. When an element is to be inserted in an already occupied index, **the element is simply appended to the list** in that index. Since an open dictionary is basically an array of linked lists.
- **Close dictionaries** on the other hand, linear probing OR other collision management techniques are used to manage synonyms. This is when an element is to be inserted at an index but then the bucket at that index already contains another element. Depending on what is specified, you could have any other collision management technique but most commonly, linear probing has been frequently used. **Linear probing is simply finding the next available bucket starting from the returned hash-key index**, instead of beginning the search at index zero, you begin at the indexed hash key-value.

The Concept of Load Factor and Rehashing

What is the “Load Factor”?

- Load factor is simply the ratio between the stored key-value pairs in the dictionary to the total number of buckets.
- Suppose that **n** is the number of key-value pairs, whereas **m** is the total number of buckets of the current dictionary.
 - The formula is: **load factor (x) = n / m**
- The higher the load factor, the higher the chances are for collisions. This has the tendency to slow down operations.
- The typical threshold to have is approximately equal to 0.7-1.0, after which the hash table is resized.

You may be wondering...“after which the hash table is resized”?

Yep, dictionary enables resizing when the data structure demands it. This is to maintain the threshold’s advised load fact

Open Dictionary / Open Hashing / External Hashing

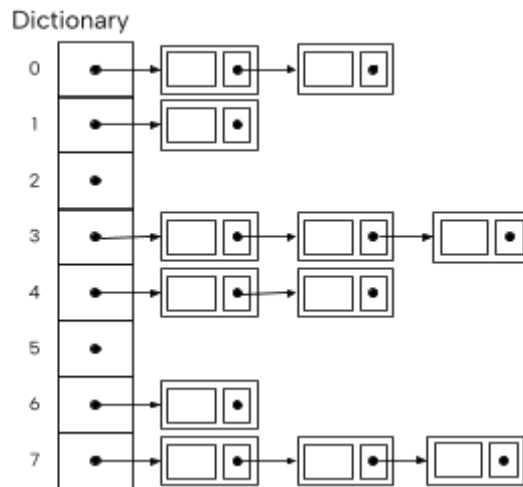
An open dictionary, often referred to as hash tables with chaining, is a dictionary data structure implemented through an array of linked list head pointers as per each index in the array. Allowing elements in the set to be stored in potentially unlimited space. **Synonyms are stored on the same index or bucket by appending to the list per each index.**

What does this look like in code?

- The following is a code snippet of how an open dictionary is typically declared.
- Followed by a visualization of the provided code afterwards.

```
#define MAX 8
```

```
typedef struct node { // similar structure to a linked list structure
    int data;
    struct node *link;
} *SETPTR;
typedef SETPTR Dictionary[MAX]; // the array of head pointers
```



In **Chaining**, as we've previously mentioned, collisions are resolved by storing multiple key-value pairs in a linked list at the same index (bucket). Where each bucket is essentially a pointer to the head of the linked list.

Advantages of Open Hash	Disadvantage of Open Hash
Simple to implement	Memory overhead per new node created for each element
Can accommodate a large number of key-value pairs, even if the number of keys exceeds the number of buckets	Cache performance cost due to non-contiguous memory access when traversing linked lists
Performance is dependent to the number of elements in the dictionary; increases alongside its Load Factor	Worst time complexity for operations to be $O(n)$ if many keys hash to the same bucket

Closed Dictionary / Closed Hashing / Internal Hashing

A closed dictionary is a dictionary data structure implemented through an array where each index accommodates one element only. This uses a fixed space for storage, limiting the size of the set. **Synonyms enable collision management such as 2-pass loading, linear probing, and the like.** This is when elements have the same hash key value and following elements with synonyms are displaced in the dictionary.

This is simply an array of buckets in which each bucket IS LIMITED to store one element only; the data structure best to represent this is an array list. The **array bucket attains the structure of an array but we will treat it as a circular array** to keep traversal in a consistent direction using modulo operation to the size of the array itself.

Why must we use the modulo operator?

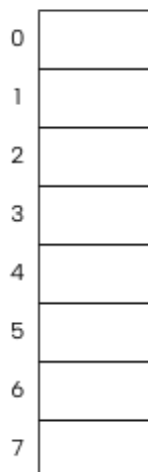
- Traditionally, without using the modulo operator at all, suppose that the hash function returns a key somewhere in the dictionary such as three (3).
- If the element to search is not stored at the third bucket then we proceed to traverse until the end of the dictionary which is at index seven (7).
- Yes, we have traversed the dictionary from the starting index to the last index but that doesn't mean we've checked and visited all buckets in the dictionary.
- To be able to search the buckets before index three (3), we use modulo (%) as we increment our counter variable to be able to circularly visit each bucket in the dictionary starting from the hash key index.

What does this look like in code?

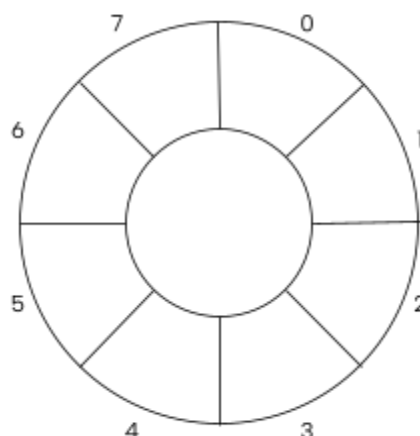
- The following is a code snippet of how an open dictionary is typically declared.
- Followed by a visualization of the provided code afterwards.

```
#define MAX 8
#define EMPTY '~' // state of a bucket
#define DELETED '!' // state of a bucket
typedef char Dictionary[MAX];
```

How the dictionary looks **PHYSICALLY**



How the dictionary looks **LOGICALLY**



Collision Management

Collision Management or resolution techniques in a dictionary (such as a hash table) can vary depending on the program's specifications and requirements. However, they all belong to a broader category of strategies designed to handle collisions—situations where multiple keys (often referred to as "synonyms" in this context, meaning keys that produce the same hash value) map to the same bucket or index as an existing key-value pair in the dictionary.

- In open dictionaries using chaining, for example, this is typically resolved by appending the new key-value pair to a linked list at the collided bucket, ensuring all synonyms are stored and accessible without overwriting data.
- Other approaches, like open addressing in closed dictionaries, probe for alternative slots instead.
- The choice of technique balances factors such as performance, memory usage, and load factor to maintain efficient operations.

Exercise: Write the code of the following

1. The hash() function accepts as parameters an integer x and returns the digit in the hundredths place of x.
2. The hash() function accepts as parameter an integer x and returns a hash value between 0–18. The hash value is the remainder when the sum of all the digits in x is divided by 19.
3. The hash() function accepts as parameter a name and returns a hash value between 0–48. The hash value is the remainder when the sum of the ASCII values of all the letters in the name is divided by 49.
4. Implement an open dictionary where each bucket is sorted and tracks the number of collisions per bucket. For each bucket, maintain a count of key-value pairs (synonyms). When inserting, if a bucket's collision count exceeds a threshold (e.g., 3), split the bucket's contents into two new buckets by rehashing with a secondary hash function.
5. **OPTIONAL:** Implement an open dictionary where each bucket uses a binary search tree (BST) instead of a linked list to store colliding keys. This reduces search time for buckets with many collisions to $O(\log k)$ instead of $O(k)$, where k is the number of synonyms.
6. **OPTIONAL:** Implement an open dictionary where each bucket uses a Partially Ordered Tree (POT) in Min Heap instead of a linked list to store colliding keys. Simulate insert, remove, and member.

Also try various structure definitions!

Open Dictionary	Close Dictionary
<pre>#define MAX 8 typedef struct node { int key; char* value; struct node* next; } *Dictionary[MAX];</pre>	<pre>#define MAX 8 typedef struct { int key; char* value; char status; // 'E' for empty, 'D' for deleted, 'O' for occupied } Bucket; typedef Bucket Dictionary[MAX];</pre>

End. Happy learning!